

Vision System Documentation

Educational Robot — Computer Vision Pipeline & Module Architecture

1. Folder Purpose

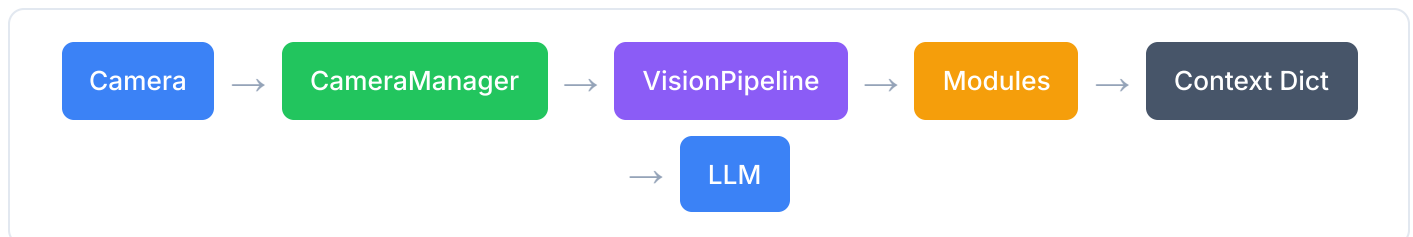
The `vision/` package provides the complete computer vision subsystem for an educational robotics platform. It is responsible for capturing live camera frames, processing them through a pipeline of specialised modules, and producing a structured **context dictionary** that is consumed by an LLM-based reasoning layer. The system supports:

- **Face tracking** — identity recognition, bounding box, landmark estimation
- **Gesture recognition** — hand pose and motion classification
- **Emotion detection** — facial-expression-based affect classification
- **Object detection** — bounding boxes and labels for known object classes
- **Scene understanding** — semantic segmentation and spatial layout analysis
- **Obstacle avoidance** — depth-aware obstacle proximity estimation

Target Platforms: Raspberry Pi 4/5 (with Picamera2) and x86_64 desktops (USB camera via OpenCV). The pipeline automatically adapts its module set and performance parameters to the hardware profile.

2. Architecture Overview

The vision system follows a linear **orchestrator pipeline pattern**. A single `VisionPipeline` class owns a `CameraManager` instance and a set of sub-modules. At each tick it captures a frame, runs the modules in a defined order, and merges their outputs into a single dictionary. That dictionary is made available to the LLM reasoning layer via a thread-safe shared context.



The pipeline runs in a dedicated background thread, decoupling the camera frame-rate from the LLM inference cadence. Fresh context is produced continuously and stored in a ring buffer; the LLM consumer retrieves the most recent complete context snapshot on demand.

3. Files Overview

File	Lines	Role	Key Classes / Functions
<code>__init__.py</code>	1	Package marker	—
<code>camera.py</code>	~176	Camera abstraction layer	<code>CameraManager</code>
<code>pipeline.py</code>	~418	Orchestrator & context assembly	<code>VisionPipeline</code>

A further set of module implementations live under `vision/modules/` (not documented in detail here):

Module	Class	Description
<code>face_tracker.py</code>	<code>FaceIdentityTracker</code>	Face detection, embedding, identity matching, landmarks
<code>gesture.py</code>	<code>GestureDetector</code>	Hand landmark extraction and gesture classification
<code>emotion.py</code>	<code>EmotionDetector</code>	Facial expression → emotion category mapping
<code>objects.py</code>	<code>ObjectRecognitionModule</code>	Object bounding box detection and labelling
<code>scene.py</code>	<code>SceneSegmentationModule</code>	Semantic segmentation and layout parsing
<code>obstacle.py</code>	<code>ObstacleDetector</code>	Depth-based obstacle proximity detection

4. Detailed File Explanations

4.1 `camera.py` — `CameraManager`

`CameraManager` is the sole point of access for camera hardware. It provides a dual-backend strategy:

- **Picamera2** (Raspberry Pi) — uses the official `picamera2` library with a preview configuration at the configured resolution (`width × height`) and framerate.
- **OpenCV** (fallback / desktop) — uses `cv2.VideoCapture` with an ordered list of `fallback_indices` . It attempts each index in sequence until one succeeds.

Key design points:

- Backend is selected at construction time based on `IS_RASPBERRY_PI` (injected from `config.settings`).
- On failure, `open()` returns `False` and marks the camera as unavailable; the pipeline degrades gracefully by producing a default context with an error message.
- `get_frame()` returns a `numpy.ndarray` (BGR) or `None` ; frames are flipped vertically via OpenCV after capture (Picamera2 handles orientation natively).
- The class implements the context manager protocol (`__enter__` / `__exit__`) for safe resource cleanup.
- A `stream_buffer_size` parameter controls the internal deque size used by the streaming generator (`start_stream`).

Fig 4.1 — CameraManager Backend Selection & Fallback

[Screenshot: constructor branching on `IS_RASPBERRY_PI`, fallback loop in `_open_opencv`]

Fig 4.2 — OpenCV Camera Opening with Retry Indices

[Screenshot: `_open_opencv` iterating `FALLBACK_INDICES`, calling `cv2.VideoCapture`, checking `isOpened()`]

4.2 `pipeline.py` — VisionPipeline

`VisionPipeline` is the orchestrator. It initialises, configures, and runs all vision sub-modules. It owns a `CameraManager` and manages a background processing thread.

Initialisation

The constructor allocates all sub-module references as `None` , creates a `deque` context buffer (maxlen = 10), and sets counters for frame-skip and thermal state tracking. No hardware resources are acquired until `open()` is called.

`open()` — Start-Up Sequence

1. Call `camera.open()` — acquire camera hardware.

- 2. If the camera is available, instantiate each sub-module (`FaceIdentityTracker` , `GestureDetector` , `EmotionDetector` , `ObjectRecognitionModule` , `SceneSegmentationModule` , `ObstacleDetector`).
- 3. Apply the current vision profile (`MINIMAL` , `BALANCED` , or `FULL`) — this enables or disables modules and adjusts frame-skip targets.
- 4. Call `warmup(frames=3)` to allow auto-exposure and face-detection caches to settle before producing useful context.
- 5. Start the background processing thread via `start()` .

Fig 4.3 — VisionPipeline.open() with Module Initialisation

[Screenshot: camera open, module instantiations, apply_profile, warmup, thread start]

Profile System

Three profiles are defined, each suited to a different hardware capability tier:

Profile	Active Modules	Target Use	Frame Skip
MINIMAL	Face tracker only	Raspberry Pi Zero / low-power	3 (every 4th frame)
BALANCED	Face tracker, gesture, emotion	Raspberry Pi 4 / medium desktop	2 (every 3rd frame)
FULL	All six modules	Raspberry Pi 5 / desktop GPU	1 (every 2nd frame)

Profiles can be switched at runtime via `set_profile(profile)` , enabling dynamic reconfiguration without a restart. The profile is also read from settings and applied automatically during `open()` .

Fig 4.4 — Profile Application for Different Hardware

[Screenshot: _apply_profile method, conditional enabling of modules based on profile enum]

`get_all_context()` — The Main Processing Function

This is the core pipeline tick. It is called by the background thread on each iteration (subject to frame-skip). The execution order is:

1. Capture a frame via `camera.get_frame()`. If the frame is `None`, return a default context with an error entry.
2. Run **face tracker** — always executed regardless of profile (it is the most critical module for an interactive educational robot).
3. If a face was detected in the current frame, run **gesture** and **emotion** detection (these are face-relative).
4. Run **object detection**, **scene segmentation**, and **obstacle detection** in parallel (each operates on the full frame independently).
5. Merge all results into a single dictionary (using `{**face, **gesture, ...}` pattern).
6. Evaluate the “best context” heuristic (see below) and store it in the ring buffer.
7. Apply adaptive throttling based on frame processing time and CPU temperature.

Fig 4.5 — `get_all_context()` — The Main Processing Function

[Screenshot: full method body showing frame capture, module calls, context merging]

Adaptive Throttling & Thermal Management

The `_adaptive_throttle()` method implements two independent back-pressure mechanisms:

- **Thermal throttling** — reads CPU temperature via `get_cpu_temperature()` (from `config.diagnostics`). When temperature exceeds `THERMAL_THROTTLE_C` (75°C), the frame-skip counter is doubled. It is restored when temperature drops below `THERMAL_RESTORE_C`.
- **Load shedding** — measures the wall-clock time taken by the previous `get_all_context()` call. If it exceeds the target inter-frame period (derived from `CAMERA_FPS`), the frame-skip is incremented; if there is spare time, the skip is decremented (bounded to profile minimum).

Fig 4.6 — Adaptive Frame Skipping and Thermal Throttling

[Screenshot: `_adaptive_throttle` method, temperature check, frame-time comparison]

Best-Context-from-Buffer Algorithm

The pipeline maintains a `deque` of previous context dictionaries (maxlen = 10). When selecting the “best” context to expose to consumers, it scores each buffered entry based on a weighted heuristic:

- Whether a face is present (`+50`)

- Face confidence score (+0 - 30)
- Number of detected objects (+0 - 20)
- Whether gesture data is available (+15)
- Frame timestamp freshness weight

The highest-scoring entry is placed into the shared context for LLM consumption. This ensures that even if the current frame produced poor results (e.g., a motion blur), a recent high-quality context is still available.

Fig 4.7 — Best-Context-from-Buffer Algorithm

[Screenshot: deque iteration, score computation, max selection]

Default Context Structure

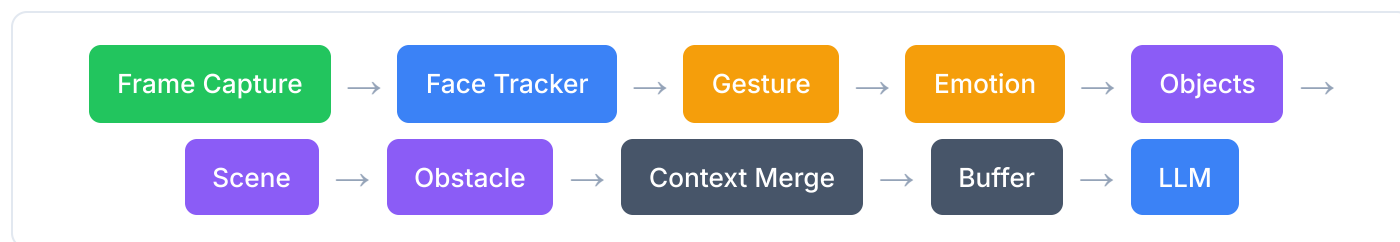
When no frame is available or the pipeline has not yet warmed up, a default context dictionary is returned. Its structure mirrors the real context so that LLM-facing code never has to handle missing keys:

```
{
  "frame": null,
  "timestamp": <float>,
  "face_detected": False,
  "face_bbox": None,
  "face_landmarks": None,
  "face_identity": None,
  "face_confidence": 0.0,
  "gesture": None,
  "gesture_confidence": 0.0,
  "emotion": None,
  "emotion_confidence": 0.0,
  "objects": [],
  "scene_layout": None,
  "obstacle_proximity": None,
  "fps": 0.0,
  "error": "No frame available"
}
```

Fig 4.8 — Default Context Structure[Screenshot: `_default_context()` method with overrides parameter]

5. Sequence Flow

The following diagram captures the end-to-end data flow within a single pipeline tick:



- 1. Frame capture** — `CameraManager.get_frame()` reads one raw BGR frame from the active backend. Resolution and FPS are governed by `config.settings.CAMERA_*` values.
- 2. Face tracking** — The `FaceIdentityTracker` runs detection + embedding on the full frame. It returns bounding boxes, landmarks, an identity string (if match found), and a confidence score.
- 3. Gesture & Emotion (conditional)** — If at least one face was found, the `GestureDetector` and `EmotionDetector` receive the face bounding box region or the full frame (module-dependent). Both return a label and confidence.
- 4. Object recognition** — `ObjectRecognitionModule` runs a pretrained object detector (YOLO / SSD) on the full frame, returning a list of `(label, bbox, confidence)` tuples.
- 5. Scene segmentation** — `SceneSegmentationModule` performs semantic pixel-wise segmentation, returning a mask array and a high-level scene label (e.g., `"indoor_classroom"`, `"outdoor_playground"`).
- 6. Obstacle detection** — `ObstacleDetector` analyses depth information (monocular depth estimation or dedicated depth sensor) to return proximity values for the nearest obstacles in the robot's path.
- 7. Context merge & buffer** — All module outputs are merged into a single flat dictionary. The buffer scoring heuristic selects the best entry for sharing.
- 8. LLM consumption** — The LLM reasoning layer calls `pipeline.get_shared_context()` which returns the current best context snapshot, lock-free (an `atomic` reference swap ensures thread safety).

6. Folder Integration

The `vision/` package does not operate in isolation. It integrates with the broader robot software stack through several interfaces:

Integration	Mechanism	Direction
Vision → LLM	<code>pipeline.get_shared_context()</code> returns the best context dict, which is injected into the LLM prompt template as structured context.	Outbound
Vision → Voice Pipeline	Face identity and emotion data may trigger proactive speech (e.g., "Hello {name}!" if a known face is detected).	Outbound
Vision → Config	<code>config.settings</code> provides camera parameters, profile selection, fallback indices, and hardware flags (<code>IS_RASPBERRY_PI</code>).	Inbound
Vision → Diagnostics	<code>config.diagnostics.get_cpu_temperature()</code> supplies thermal data for adaptive throttling.	Inbound
Vision → Dashboard	Frame rate statistics, active profile, thermal state, and per-module latency may be exposed via a debug API for real-time monitoring.	Outbound

7. Technical Notes

7.1 OpenCV

OpenCV (`cv2`) serves as the universal fallback backend and is also used for image manipulation (resizing, flipping, colour space conversion) regardless of the capture backend. The pipeline assumes BGR colour ordering throughout.

7.2 Picamera2

On Raspberry Pi hardware, the `picamera2` library provides lower latency and hardware-accelerated encoding compared with OpenCV's V4L2 backend. The preview configuration is used at the resolution and framerate specified in settings. If `picamera2` is not installed (or import fails), the system falls back to OpenCV automatically.

7.3 Watershed Pattern

The “watershed” analogy applies to the way the pipeline separates concerns: each module operates on its own “catchment area” of the input frame (full-frame for objects/scene/obstacle, face-region for gesture/emotion) and drains its output into the central context buffer. This avoids coupling between modules and allows them to be enabled, disabled, or replaced independently.

7.4 Threading

The pipeline runs in a single dedicated thread (not a thread pool) to avoid GIL contention in CPython. Context sharing with the LLM thread uses an `atomic` reference swap — the background thread writes to a private buffer and swaps a pointer; the reader reads the pointer and copies the dict. This is lock-free in practice because dict reads are atomic in CPython (implementation-specific).

7.5 Context Managers

Both `CameraManager` and `VisionPipeline` implement `__enter__` / `__exit__` so they can be used in `with` statements, ensuring that camera resources are released even if an exception occurs:

```
with VisionPipeline() as pipeline:
    pipeline.open()
    context = pipeline.get_shared_context()
```

8. Academic Explanation — Design Rationale

8.1 Why an Orchestrator Pipeline?

A monolithic “one function does everything” approach would be brittle and hard to maintain. The orchestrator pattern decouples the capture layer (`CameraManager`) from the processing layer (`VisionPipeline`) from individual vision tasks (the modules). This means:

- Each module can be developed, tested, and profiled in isolation.
- A failure in one module (e.g., gesture model fails to load) does not crash the pipeline — it simply produces a `None` output for that key.
- Adding a new vision capability requires writing a new module class and registering it in the pipeline; no other code changes are needed.

8.2 Profile System — RPi vs Desktop

The dual-platform requirement (Raspberry Pi for cost-optimised deployment, x86_64 desktop for development and high-performance scenarios) drove the design of the profile system:

- **MINIMAL** — targets edge devices with limited compute; only face tracking runs, and frame-skip is aggressive (every 4th frame processed).
- **BALANCED** — the default for RPi 4; adds gesture and emotion, which are face-region-only and therefore cheap, plus moderate frame-skip (every 3rd frame).
- **FULL** — for desktop GPUs or RPi 5 with a neural compute stick; all six modules run with minimal frame-skip.

The profile abstraction allows the same codebase to serve both deployment targets without conditional logic scattered throughout the pipeline. Runtime switching makes it possible to “turbo” the system during critical interactions and “eco” mode during idle periods.

8.3 Adaptive Performance — Why It Matters

An educational robot operates in unpredictable environments. Classroom lighting changes, the robot may overheat on a sunny windowsill, or a computationally expensive face may enter the frame. Fixed frame-skip and module sets cannot handle all scenarios. The adaptive throttling system:

1. **Prevents thermal shutdown** — by shedding load at 75°C, the system maintains functionality (at reduced fidelity) rather than crashing.
2. **Maintains latency bounds** — load shedding keeps per-frame latency under the target inter-frame period, preventing the context from becoming stale.
3. **Recovers automatically** — when temperature drops or frame time improves, the system gradually restores full operation.

8.4 Best-Context-from-Buffer — Robustness to Transient Failures

Camera frames on low-cost hardware are noisy. Motion blur, auto-exposure transitions, and partial occlusions can produce a single bad frame. The buffer-based best-context algorithm provides temporal smoothing: a brief dropout or blurry frame does not propagate to the LLM. The score heuristic prioritises frames with faces and high detection confidence, which aligns with the robot's primary interaction goal (talking to people).

8.5 Future Improvements

- **Hardware acceleration** — offload object detection and segmentation to a Coral TPU or NVIDIA Jetson GPU via TensorRT.
- **Multi-camera support** — a stereo pair or separate wide-angle / telephoto cameras could feed different modules.
- **Predictive frame skipping** — use a lightweight motion estimator to skip frames that are near-duplicates of the previous frame, saving compute.
- **Module priority scheduling** — assign a priority and budget per module (e.g., face tracking gets 50 ms, objects gets 100 ms) and abort modules that exceed their budget.

- **Event-driven triggering** — instead of polling at a fixed rate, trigger context capture on motion events or voice activity detection.

Document Version: 1.0 — Generated for graduation-project submission.